

# Esame Programmazione di Sistema / API Programming

Data: 2023-01-15

## QUESTIONS

Nel caso di programmazione concorrente in Rust e memoria condivisa quali sono i principali costrutti e come vengono utilizzati per evitare la condizione di deadlock?

Come si utilizza il sistema di gestione dei processi in Rust attraverso il pacchetto `std::process`? Quali sono le funzionalità offerte dalla libreria e come si gestiscono i processi figli nell'ambito del proprio programma?

Definire il problema delle dipendenze cicliche, includendo il motivo per cui si possa generare, ed indicare come in Rust sia possibile risolverle, facendo un esempio concreto.

Una barriera è un costrutto di sincronizzazione usato per regolare l'avanzamento relativo della computazione di più thread.

All'atto della costruzione di questo oggetto, viene indicato il numero N di thread coinvolti. Non è lecito creare una barriera che coinvolga meno di 2 thread.

La barriera offre un solo metodo, `wait()`, il cui scopo è bloccare temporaneamente l'esecuzione del thread che lo ha invocato, non ritornando fino a che non sono giunte altre N-1 invocazioni dello stesso metodo da parte di altri thread: quando ciò succede, la barriera si sblocca e tutti tornano. Successive invocazioni del metodo `wait()` hanno lo stesso comportamento: la barriera è ciclica.

Attenzione a non mescolare le fasi di ingresso e di uscita!

Una **RankingBarrier** è una versione particolare della barriera in cui il metodo `wait()` restituisce un intero che rappresenta l'ordine di arrivo: il primo thread ad avere invocato `wait()` otterrà 1 come valore di ritorno, il secondo thread 2, e così via. All'inizio di un nuovo ciclo, il conteggio ripartirà da

:9/23 7:58 PM

Pagina

1.

**Si implementi la struttura dati `RankingBarrier` a scelta nei linguaggi Rust o C++ '11 o successivi.**

# SOLUTIONS

## Domanda 1: Deadlock in Rust con Memoria Condivisa

Nella programmazione concorrente in Rust basata sulla memoria condivisa, i principali costrutti utilizzati sono:

- **Mutex<T>**: Garantisce la mutua esclusione permettendo ad un solo thread alla volta di accedere al dato protetto.
- **RwLock<T>**: Simile al Mutex, ma ottimizzato per consentire letture multiple in concorrenza (molti lettori, un solo scrittore).
- **Arc<T>** (Atomic Reference Counted): Permette a più thread di condividere la proprietà (ownership) della stessa istanza di una struttura dati (come un Mutex o RwLock) in modo thread-safe.

Il sistema di tipi di Rust garantisce, a tempo di compilazione, l'assenza di *data race*, ma **non previene i deadlock**. Per evitare la condizione di deadlock, il programmatore deve adottare strategie architetturali:

1. **Ordinamento delle lock (Lock ordering)**: Quando un thread ha bisogno di acquisire più lock contemporaneamente, deve farlo sempre nello stesso ordine rigoroso rispetto a tutti gli altri thread nel sistema.
2. **Try-locking**: Utilizzare metodi come `try_lock()`, che invece di bloccare indefinitamente il thread se la lock è occupata, ritornano un `Err`. Questo permette al thread di rilasciare eventuali altre lock già acquisite, aspettare e riprovare in seguito, evitando così un "circular wait".
3. **Minimizzare la scope delle lock**: Poiché in Rust il lock viene rilasciato automaticamente quando il `MutexGuard` esce dallo scope (grazie al tratto `Drop`), inserire l'acquisizione del lock in blocchi ristretti limita il tempo in cui la risorsa è trattenuta e abbassa la probabilità di overlapping tra lock diversi.

## Domanda 2: Gestione dei processi con `std::process`

Il modulo `std::process` in Rust permette di creare e gestire processi esterni al programma in esecuzione. Il componente principale è il builder `Command`.

**Funzionalità offerte:**

- **Lancio di un eseguibile**: Si utilizza `Command::new("nome_eseguibile")`.
- **Configurazione**: È possibile passare argomenti tramite `.arg()` o `.args()`, modificare le variabili d'ambiente con `.env()` o `.env_clear()`, e definire la directory di lavoro.
- **Gestione dell'I/O**: Si possono reindirizzare standard input, output ed error configurandoli con `Stdio::piped()` (per poterli leggere/scrivere dal processo padre), `Stdio::null()`, o `Stdio::inherit()`.

**Gestione dei processi figli**: Un processo può essere avviato in modalità sincrona o asincrona (dal punto di vista del padre):

- `.output()`: Esegue il processo bloccando il padre fino al termine, ritornando un oggetto contenente status code e buffer di stdout e stderr.
- `.status()`: Esegue il processo bloccando il padre fino al termine, ma ritorna unicamente lo status code d'uscita (l'output viene stampato direttamente sul terminale di default).
- `.spawn()`: Esegue il processo in background in modo non bloccante. Ritorna una struttura `Child` che rappresenta il processo figlio in esecuzione. Il padre può interagire con gli stream I/O del figlio in tempo reale e, una volta terminato, invocare obbligatoriamente `child.wait()` per attendere la terminazione del figlio e raccoglierne l'exit status, prevenendo la formazione di processi zombie.

### Domanda 3: Il problema delle dipendenze cicliche e come risolverle in Rust

Una dipendenza ciclica si verifica quando due o più strutture dati si referenziano a vicenda creando un loop chiuso (ad esempio, l'oggetto A contiene un riferimento all'oggetto B, che a sua volta ha un riferimento ad A). Nei sistemi che si appoggiano allo *smart pointer* con conteggio dei riferimenti (come `Rc<T>` e `Arc<T>`), questo causa un **memory leak**: le variabili usciranno di scope, ma il conteggio dei riferimenti interni non scenderà mai a zero. Di conseguenza, i dati non verranno mai deallocati dalla memoria.

**Risoluzione in Rust:** Per rompere un ciclo di dipendenze, Rust offre gli **Weak Pointers** (`std::rc::Weak` o `std::sync::Weak`). Un puntatore debole fa riferimento ai dati senza incrementare lo "strong reference count". Se tutti gli strong pointers vengono deallocati, il dato viene distrutto indipendentemente dai weak pointers, i quali restituiranno semplicemente un `None` al tentativo di accesso.

**Esempio concreto (Struttura ad Albero con riferimento al padre):**

```
use std::rc::{Rc, Weak};
use std::cell::RefCell;

struct Node {
    value: i32,
    // Weak evita il ciclo di dipendenze tra figlio e padre
    parent: RefCell<Weak<Node>>,
    // Rc possiede in maniera "forte" i figli
    children: RefCell<Vec<Rc<Node>>>,
}

fn main() {
    let parent = Rc::new(Node {
        value: 1,
        parent: RefCell::new(Weak::new()), // Radice senza padre
        children: RefCell::new(vec![]),
    });

    let child = Rc::new(Node {
        value: 2,
        // Usiamo Rc::downgrade per creare un Weak pointer al padre
        parent: RefCell::new(Rc::downgrade(&parent)),
        children: RefCell::new(vec![]),
    });

    parent.children.borrow_mut().push(child);

    // Al termine dello scope, parent scende a count 0
    // e dealloca tutta la gerarchia correttamente.
}
```

### Domanda 4: Implementazione di RankingBarrier

Per implementare `RankingBarrier` utilizziamo un `Mutex` a protezione dello stato interno (un counter per sapere quanti thread sono arrivati) e una `Condvar` per sospendere e risvegliare l'esecuzione. In aggiunta, per prevenire *mix delle fasi* in barrier cicliche, manteniamo un identificativo del ciclo attuale (`generation`). In questo modo, un thread molto veloce risvegliato per un ciclo successivo non può influire sui thread ancora da risvegliare nel ciclo precedente.

```
use std::sync::{Mutex, Condvar};

pub struct RankingBarrier {
    n: usize,
    state: Mutex<BarrierState>,
    cvar: Condvar,
}

struct BarrierState {
    count: usize,
```

```

        generation: usize,
    }

    impl RankingBarrier {
        /// Inizializza la barriera con N thread
        pub fn new(n: usize) -> Self {
            assert!(n >= 2, "La barriera deve coinvolgere almeno 2 thread");
            Self {
                n,
                state: Mutex::new(BarrierState {
                    count: 0,
                    generation: 0,
                }),
                cvar: Condvar::new(),
            }
        }

        /// Blocca il thread fino all'arrivo di tutti gli N thread.
        /// Restituisce l'ordine di arrivo per il ciclo corrente (da 1 a N).
        pub fn wait(&self) -> usize {
            let mut state = self.state.lock().unwrap();

            // Salviamo la generazione corrente in una variabile locale
            let local_gen = state.generation;

            // Questo thread e' arrivato
            state.count += 1;
            let rank = state.count;

            if state.count == self.n {
                // Sono l'ultimo thread, devo sbloccare tutti
                state.count = 0;
                // Avanza il ciclo
                state.generation = state.generation.wrapping_add(1);
                self.cvar.notify_all();
            } else {
                // Non sono l'ultimo, attendo che la generazione cambi
                while local_gen == state.generation {
                    state = self.cvar.wait(state).unwrap();
                }
            }

            // Restituisco l'ordine di arrivo calcolato all'ingresso
            rank
        }
    }
}

```